Dynamic Programming II

Knapsack, LIS, LCS, edit distance and grid DP

Learning goals

- Design DP with **two-dimensional state** (item × capacity, prefix × prefix, row × column)
- Solve 0/1 and unbounded knapsack, and compress them to one dimension
- Compute LIS in $O(n^2)$ and in $O(n \log n)$
- Solve LCS and edit distance, and **reconstruct** an optimal answer
- Count paths in a grid with obstacles
- Recognise these five patterns inside unfamiliar problems

Recap: the DP recipe (Week 14)

1. **State**: what does `dp[...]` mean, in one sentence?
2. **Transition**: how is a state built from smaller states?
3. **Base cases**: the smallest states, written down directly
4. **Order**: evaluate so every dependency is ready first
5. **Answer**: which state (or max over states) is the answer?

Complexity ≈ (number of states) × (cost per transition)

Motivating problem: Book Shop (CSES 1158)

- n books, book i costs `h[i]` and has `s[i]` pages
- Budget `x`. Maximise total pages; each book bought **at most once**
- $n \le 1000$, $x \le 10^5$

Greedy by pages/price fails:

book	price	pages	ratio
A	6	7	1.17
B	5	5	1.00
C	5	5	1.00

Budget 10: greedy takes A (7 pages), optimum B+C (10 pages).

0/1 knapsack: state and transition

`dp[i][c]` = max pages using only the first `i` books with total cost ≤ `c`

```
dp[i][c] = max( dp[i-1][c],                    // skip book i
                dp[i-1][c - h[i]] + s[i] )     // take book i (if c >= h[i])
```

- Base: `dp[0][c] = 0` for all c
- Answer: `dp[n][x]`
- States: n·x = 10^8 … transitions O(1) → too much memory as a 2D table

Rolling to one dimension

Row `i` only reads row `i-1`. Keep one array and overwrite it.

```
vector<int> dp(x + 1, 0);
for (int i = 0; i < n; i++)
    for (int c = x; c >= h[i]; c--)      // DESCENDING
        dp[c] = max(dp[c], dp[c - h[i]] + s[i]);
```

- Descending `c`: `dp[c - h[i]]` is still the **old** row → book used once
- Ascending `c` would reuse book `i` many times → that is **unbounded** knapsack
- Memory O(x), time O(n·x)

Knapsack trace (tiny)

Items (cost, value): (2,3), (3,4). Capacity 5.

c	0	1	2	3	4	5
start	0	0	0	0	0	0
after (2,3)	0	0	3	3	3	3
after (3,4)	0	0	3	4	4	**7**

Ascending order with (2,3) would give `dp[4] = 6` — item taken twice.

Knapsack family

Variant	Loop over capacity	Example
0/1 (max value)	descending	CSES 1158 Book Shop
Unbounded	ascending	CSES 1636 Coin Combinations II
Counting subsets	descending, `+=`	CSES 1093 Two Sets II
Reachable sums	descending, `	=` (bitset!)
Small cap, big values	swap roles: `dp[value] = min cost`	AtCoder DP E

Longest Increasing Subsequence

Given `a[0..n-1]`, longest strictly increasing subsequence (not contiguous).

`7 3 5 3 6 2 9 8` → `3 5 6 9`, length 4

O(n^2) DP: `dp[i]` = length of the LIS **ending at** i

```
dp[i] = 1 + max{ dp[j] : j < i, a[j] < a[i] }   (or 1 if none)
answer = max over i of dp[i]
```

Fine for n ≤ 5000. CSES 1145 has n = $2 \cdot 10^5$ → need better.

LIS in O(n log n): the `tails` array

`tails[k]` = smallest possible last element of an increasing subsequence of length k+1

- `tails` is always **strictly increasing**
- For each `a`: find first `tails[k] >= a` (lower_bound)
  - none → append (we extended the longest one)
  - else → replace `tails[k] = a` (same length, smaller tail)

a	7	3	5	3	6	2	9	8
tails	7	3	3 5	3 5	3 5 6	2 5 6	2 5 6 9	2 5 6 8

Answer = `tails.size()` = 4. (`tails` itself is **not** an LIS!)

LIS code

```
vector<int> tails;
for (int i = 0; i < n; i++) {
    int a; cin >> a;
    auto it = lower_bound(tails.begin(), tails.end(), a);
    if (it == tails.end()) tails.push_back(a);
    else *it = a;
}
cout << tails.size() << "\n";
```

- Strictly increasing → `lower_bound`
- Non-decreasing → `upper_bound`
- To reconstruct: store index of each tail + parent pointer per element

Longest Common Subsequence

`dp[i][j]` = LCS length of prefixes `s[0..i)` and `t[0..j)`

```
if s[i-1] == t[j-1]:  dp[i][j] = dp[i-1][j-1] + 1
else:                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

- Base: row 0 and column 0 are 0
- O(n·m) time and memory
- Dependencies: up, left, up-left → fill row by row

LCS table example: `s = axyb`, `t = abyxb`

	ε	a	b	y	x	b
ε	0	0	0	0	0	0
a	0	1	1	1	1	1
x	0	1	1	1	2	2
y	0	1	1	2	2	2
b	0	1	2	2	2	**3**

Answer 3: e.g. `axb` or `ayb`.

Reconstruction: walk back from the answer

```
string ans; int i = n, j = m;
while (i > 0 && j > 0) {
    if (s[i-1] == t[j-1]) { ans += s[i-1]; i--; j--; }
    else if (dp[i-1][j] >= dp[i][j-1]) i--;
    else j--;
}
reverse(ans.begin(), ans.end());
```

General rule: at each state, find **which transition produced the value** and follow it. Keep the full table (don't roll it) if you need to reconstruct.

Edit distance (CSES 1639)

Min number of insert / delete / replace to turn `a` into `b`.

`dp[i][j]` = edit distance between `a[0..i)` and `b[0..j)`

```
dp[i][0] = i,  dp[0][j] = j
dp[i][j] = min( dp[i-1][j]   + 1,              // delete a[i-1]
                dp[i][j-1]   + 1,              // insert b[j-1]
                dp[i-1][j-1] + (a[i-1]!=b[j-1]) )  // replace or match
```

`LOVE → MOVIE` = 2 (replace L→M, insert I). O(n·m).

Edit distance code

```
int n = a.size(), m = b.size();
vector<vector<int>> dp(n + 1, vector<int>(m + 1));
for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = min({dp[i-1][j] + 1,
                        dp[i][j-1] + 1,
                        dp[i-1][j-1] + (a[i-1] != b[j-1])});
cout << dp[n][m] << "\n";
```

Same shape as LCS — "two prefixes" is a pattern, not a single problem.

Grid DP: Grid Paths (CSES 1638)

n×n grid, `*` = trap. Count paths from top-left to bottom-right moving only **right/down**, mod 10^9+7.

`dp[i][j]` = number of ways to reach cell (i, j)

```
dp[i][j] = 0                              if (i,j) is a trap
dp[i][j] = dp[i-1][j] + dp[i][j-1]   otherwise
dp[0][0] = 1 if not a trap
```

Moves only go right/down → the grid is a DAG; row-major order is a valid topological order.

Grid DP code

```
vector<vector<int>> dp(n, vector<int>(n, 0));
if (g[0][0] == '.') dp[0][0] = 1;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        if (g[i][j] == '*') { dp[i][j] = 0; continue; }
        if (i > 0) dp[i][j] = (dp[i][j] + dp[i-1][j]) % MOD;
        if (j > 0) dp[i][j] = (dp[i][j] + dp[i][j-1]) % MOD;
    }
```

Variants: max-sum path, min-cost path, "collect coins", paths with k turns (add a dimension).

Complexity summary

Problem	States	Transition	Time	Memory
0/1 knapsack	$n \cdot W$	$O(1)$	$O(nW)$	$O(W)$ rolled
LIS (DP)	n	$O(n)$	$O(n^2)$	$O(n)$
LIS (tails)	—	$O(\log n)$	$O(n \log n)$	$O(n)$
LCS / edit distance	$n \cdot m$	$O(1)$	$O(nm)$	$O(nm)$ or $O(m)$
Grid paths	$n \cdot m$	$O(1)$	$O(nm)$	$O(nm)$ or $O(m)$

Rule of thumb: ~10^8 simple operations per second.

Spotting the pattern

- "choose a subset under a budget" → **knapsack**
- "longest chain where each next is bigger" → **LIS** (often after sorting by one key)
- "two strings / sequences, align them" → **LCS / edit distance** (prefix × prefix)
- "move right/down on a grid" → **grid DP**
- n·W too big? Check whether the **values** are small → index by value instead

Common pitfalls

- Wrong loop direction in rolled knapsack (0/1 vs unbounded)
- Using `upper_bound` vs `lower_bound` wrongly in LIS (strict vs non-strict)
- Printing `tails` as if it were the LIS
- Off-by-one between string index and DP index (`s[i-1]` for `dp[i]`)
- Forgetting the trap at the **start** cell in grid DP
- `int` overflow in sums of values — use `long long`
- Allocating a 2D table of 10^8 ints (400 MB) instead of rolling

Practice problems

- CSES 1158 Book Shop · CSES 1745 Money Sums · CSES 1093 Two Sets II
- AtCoder DP Contest D (Knapsack 1) and E (Knapsack 2)
- CSES 1145 Increasing Subsequence · CF 340D Bubble Sort Graph
- AtCoder DP Contest F (LCS) · CSES 1639 Edit Distance
- CSES 1638 Grid Paths · AtCoder DP Contest H (Grid 1)
- CSES 1744 Rectangle Cutting · CF 1350B Orac and Models (LIS-style)
- Full list with links in `notes.md`

Summary

- 2D state = "first i items × capacity" or "prefix × prefix" or "row × column"
- Rolling arrays save memory; **loop direction encodes the semantics**
- LIS: $O(n^2)$ DP, or $O(n \log n)$ with `tails` + `lower_bound`
- LCS and edit distance: three-way transitions on prefix pairs
- Reconstruction = follow the transition that produced each value
- Next week: **Graphs I** — and later (Week 21) we'll see every DP is a path in a DAG